

FIAP — Mastering Relational and Non-Relational Database

Projeto Zelo — Sistema Operacional de Cuidado Animal

Documento técnico da 3.^a Sprint

Integrantes: preencher com os nomes completos em ordem alfabética.

RMs: preencher os RMs correspondentes.

Este documento foi preparado para acompanhar o arquivo `T_CH_CHALLENGE_COMPLETO.sql`. O script contém a estrutura, a carga de dados, os objetos PL/SQL, os testes de exceção e a auditoria DML.

1. Objetivo da entrega

O projeto Zelo transforma o cuidado animal de uma abordagem reativa em uma jornada proativa e preditiva. Para a sprint de banco de dados, a modelagem foi organizada ao redor de tutores, pets, clínicas, triagens, alertas e fatos de cuidado.

A implementação prioriza o fluxo de triagem inteligente: o tutor registra um relato por texto, áudio ou imagem; o sistema associa o relato ao pet e ao tutor; a clínica pode ser indicada; e o histórico fica disponível para o acompanhamento. A tabela de fatos permite análises manuais de valores por clínica e tipo de cuidado.

2. Arquivos da entrega

Arquivo	Conteúdo
<code>T_CH_CHALLENGE_COMPLETO.sql</code>	DDL, carga de dados, duas funções, dois procedimentos, trigger, testes e consultas de conferência.
<code>T_CH_DOCUMENTACAO.pdf</code>	Documentação técnica para a entrega.

3. Convenção de nomes

As tabelas seguem o padrão `T_CH_<ENTIDADE>` . Os atributos seguem prefixos semânticos: `id_` para identificadores, `nm_` para nomes, `ds_` para descrições ou textos, `tp_` para tipos, `st_` para status, `dt_` para datas, `dh_` para data e hora, `vl_` para valores, `qt_` para quantidades, `fl_` para indicadores lógicos e `sg_` para siglas.

Todos os identificadores principais utilizam `GENERATED ALWAYS AS IDENTITY` , implementando o auto incremento nativo do Oracle.

4. Tabelas criadas

Tabela	Finalidade	Carga prevista
<code>T_CH_USUARIO</code>	Usuários tutores, veterinários e gestores.	5
<code>T_CH_TUTOR</code>	Dados específicos dos tutores.	5
<code>T_CH_CLINICA</code>	Clínicas parceiras.	5
<code>T_CH_PET</code>	Animais acompanhados.	5
<code>T_CH_PET_TUTOR</code>	Relacionamento N:N do recurso Multi-Tutor.	5
<code>T_CH_TRIAGEM</code>	Triagens realizadas pelo agente inteligente.	5
<code>T_CH_ALERTA</code>	Vacinas, check-ups, retenção e pós-consulta.	5
<code>T_CH_FATO_CUIDADO</code>	Fatos analíticos para somatórios.	10
<code>T_CH_AUDITORIA_DML</code>	Histórico das operações sobre triagens.	Gerada pelo trigger

A tabela de fatos possui dez linhas detalhadas, distribuídas por diferentes clínicas e tipos de cuidado, permitindo demonstrar as combinações, os subtotais e o total geral exigidos.

5. Relacionamentos

`T_CH_USUARIO` possui relação opcional de especialização com `T_CH_TUTOR`. A tabela `T_CH_PET_TUTOR` implementa a relação muitos-para-muitos entre pets e tutores. `T_CH_TRIAGEM` relaciona pet, tutor e clínica, formando o núcleo do pré-atendimento. `T_CH_ALERTA` relaciona pet e clínica para suportar lembretes. `T_CH_FATO_CUIDADO` relaciona clínica e pet para análise. Cada operação DML realizada em `T_CH_TRIAGEM` pode gerar uma linha em `T_CH_AUDITORIA_DML`.

6. Função 1 — conversão manual para JSON

A função `FN_T_CH_TRIAGEM_JSON` recebe `p_id_triagem` e retorna um `CLOB` com a representação textual dos dados relacionais. Ela utiliza `JOIN` entre `T_CH_TRIAGEM`, `T_CH_PET`, `T_CH_TUTOR`, `T_CH_USUARIO` e `T_CH_CLINICA`.

A montagem do JSON é feita manualmente por concatenação de strings e escape de aspas. Não são utilizadas funções Oracle de JSON, como `TO_JSON`, `JSON_OBJECT`, `JSON_VALUE`, `JSON_QUERY` ou `JSON_TABLE`.

A função trata ausência de dados com `NO_DATA_FOUND`, duplicidade inesperada com `TOO_MANY_ROWS`, erro de conversão com `VALUE_ERROR` e erro genérico com `OTHERS`.

Exemplo de execução

```
SELECT FN_T_CH_TRIAGEM_JSON(1) AS ds_json FROM DUAL;
```

O resultado esperado contém campos como `id_triagem`, `pet`, `tutor`, `clinica`, `canal`, `relato`, `score_risco`, `urgencia` e `status`.

7. Função 2 — classificação da urgência

A função `FN_T_CH_CLASSIFICAR_URGENCIA` recebe um score de risco entre 0 e 100 e um indicador de sinal de alerta, com valores 0 ou 1. A regra implementada é:

Regra	Resultado
Sinal de alerta igual a 1 ou score maior ou igual a 80	EMERGENCIA
Score entre 60 e 79,99	ALTA
Score entre 30 e 59,99	MEDIA
Score abaixo de 30	BAIXA

A função trata score nulo ou fora da faixa, indicador diferente de 0 ou 1, erro de valor e erro inesperado.

```
SELECT FN_T_CH_CLASSIFICAR_URGENCIA(88, 0) AS resultado FROM DUAL;
SELECT FN_T_CH_CLASSIFICAR_URGENCIA(35, 0) AS resultado FROM DUAL;
SELECT FN_T_CH_CLASSIFICAR_URGENCIA(10, 0) AS resultado FROM DUAL;
```

8. Procedimento 1 — JOIN e JSON

O procedimento `PR_T_CH_EXIBIR_TRIAGEM_JSON` recebe um identificador de triagem, chama a Função 1 e exibe o resultado no `DBMS_OUTPUT`. A função chamada internamente é responsável pelo `JOIN` entre as tabelas relacionais e pela conversão manual para JSON.

```
BEGIN
    PR_T_CH_EXIBIR_TRIAGEM_JSON(1);
END;
/
```

O procedimento trata falha de localização, erro de formato e erro inesperado. Para a documentação da execução, deve ser capturado um print do `DBMS_OUTPUT` mostrando a string JSON.

9. Procedimento 2 — tabela de fatos e somatórios manuais

O procedimento `PR_T_CH_RESUMO_FATOS` lê `T_CH_FATO_CUIDADO` com cursor ordenado por `id_clinica`, `tp_cuidado` e `id_fato`. A lógica procedural mantém acumuladores para:

Acumulador	Finalidade
<code>v_vl_combinacao</code>	Soma da combinação completa clínica + tipo de cuidado.
<code>v_vl_subtotal</code>	Soma de todos os cuidados da clínica atual.
<code>v_vl_total</code>	Soma geral de todos os registros.

A mudança de tipo de cuidado imprime o subtotal da combinação. A mudança de clínica imprime o subtotal da clínica. Após o cursor, o procedimento imprime o último subtotal e o total geral.

O procedimento não utiliza `ROLLUP`, `CUBE`, `GROUPING SETS` ou `GROUPING`. As somas e as mudanças de agrupamento são realizadas diretamente no corpo PL/SQL.

```
BEGIN
    PR_T_CH_RESUMO_FATOS;
END;
/
```

10. Trigger de auditoria DML

O trigger `TRG_T_CH_AUDITORIA_TRIAGEM` é definido como `AFTER INSERT OR UPDATE OR DELETE ON T_CH_TRIAGEM FOR EACH ROW`. Ele grava na tabela `T_CH_AUDITORIA_DML` o usuário da sessão, o tipo da operação, a data e hora, o nome da tabela, a chave do registro, os valores anteriores e os valores novos.

Os valores são montados como texto JSON simples para permitir a visualização do conteúdo de `:OLD` e `:NEW`, sem transformar o trigger em uma dependência de funções JSON built-in.

```
SELECT id_auditoria, nm_usuario, tp_operacao, dh_operacao,
       nm_tabela, id_registro, ds_valor_anterior, ds_valor_novo
FROM T_CH_AUDITORIA_DML
ORDER BY id_auditoria;
```

As cinco inserções de triagens realizadas no script acionam o trigger e geram registros de auditoria. O script também possui testes de atualização e exclusão que podem ser executados adicionalmente, caso seja necessário demonstrar as três operações DML.

11. Testes de exceções

O script contém testes explícitos para as situações abaixo.

Objeto	Situação testada	Tratamento
<code>FN_T_CH_TRIAGEM_JSON</code>	ID de triagem inexistente.	<code>NO_DATA_FOUND</code> .
<code>FN_T_CH_CLASSIFICAR_URGENCIA</code>	Score igual a 150.	<code>RAISE_APPLICATION_ERROR</code> .
<code>FN_T_CH_CLASSIFICAR_URGENCIA</code>	Indicador de alerta igual a 7.	<code>RAISE_APPLICATION_ERROR</code> .
<code>PR_T_CH_EXIBIR_TRIAGEM_JSON</code>	Triagem inexistente.	<code>NO_DATA_FOUND</code> e erro tratado.
<code>PR_T_CH_RESUMO_FATOS</code>	Tabela de fatos vazia.	Erro de regra de negócio.
<code>PR_T_CH_RESUMO_FATOS</code>	Conversão numérica inválida.	<code>VALUE_ERROR</code> .

Os testes imprimem as mensagens com `DBMS_OUTPUT` , permitindo capturar os prints exigidos na documentação.

12. Consultas de conferência

Ao final da execução, o script apresenta a contagem de registros de todas as tabelas:

```
SELECT 'T_CH_USUARIO' AS nm_tabela, COUNT(*) AS qt_registros FROM T_CH_USU
SELECT 'T_CH_TUTOR', COUNT(*) FROM T_CH_TUTOR UNION ALL
SELECT 'T_CH_CLINICA', COUNT(*) FROM T_CH_CLINICA UNION ALL
SELECT 'T_CH_PET', COUNT(*) FROM T_CH_PET UNION ALL
SELECT 'T_CH_PET_TUTOR', COUNT(*) FROM T_CH_PET_TUTOR UNION ALL
SELECT 'T_CH_TRIAGEM', COUNT(*) FROM T_CH_TRIAGEM UNION ALL
SELECT 'T_CH_ALERTA', COUNT(*) FROM T_CH_ALERTA UNION ALL
SELECT 'T_CH_FATO_CUIDADO', COUNT(*) FROM T_CH_FATO_CUIDADO UNION ALL
SELECT 'T_CH_AUDITORIA_DML', COUNT(*) FROM T_CH_AUDITORIA_DML;
```

Os valores esperados são cinco registros nas sete tabelas principais de cadastro e relacionamento, cinco triagens, cinco alertas, dez fatos e pelo menos cinco eventos de auditoria gerados pelas inserções de triagem.

13. Ordem de execução

O arquivo deve ser executado integralmente em uma conexão Oracle com permissão para criar tabelas, funções, procedimentos e triggers. A ordem já está organizada: tabelas, trigger, carga, funções, procedimentos, testes e consultas finais.

Para documentação, recomenda-se executar o script com `SERVEROUTPUT` habilitado e capturar prints de: Função 2, Procedimento 1, Procedimento 2, testes de exceção, consulta da auditoria e contagem de registros.

14. Checklist da rubrica

Item da rubrica	Situação no arquivo
Dois procedimentos	Atendido: <code>PR_T_CH_EXIBIR_TRIAGEM_JSON</code> e <code>PR_T_CH_RESUMO_FATOS</code> .
Duas funções	Atendido: conversão manual JSON e classificação de urgência.
Um trigger de auditoria	Atendido: auditoria DML em triagens.
JOIN entre tabelas	Atendido no Procedimento 1 e na Função 1.
JSON manual	Atendido por concatenação de strings.
Três ou mais exceções	Atendido em cada função e procedimento.
Tabela de fatos	Atendido com duas categorias e medida numérica.
Subtotais e total geral manuais	Atendido sem recursos automáticos de agrupamento.
Mínimo de cinco registros	Atendido nas tabelas utilizadas e superado na tabela de fatos.
Valores <code>:OLD</code> e <code>:NEW</code>	Atendido no trigger de auditoria.
Código comentado	Atendido com comentários por seção e objeto.
Arquivo SQL completo	Atendido pelo arquivo <code>T_CH_CHALLENGE_COMPLETO.sql</code> .

15. Observação sobre validação

A estrutura e o conteúdo do arquivo foram revisados estaticamente. A execução efetiva depende de um ambiente Oracle Database, pois o sandbox de preparação não possui o cliente `sqlplus` configurado. Os prints finais devem ser obtidos no Oracle SQL Developer, LiveSQL ou ambiente Oracle equivalente após a execução do script.